VisionCount AI – Problem Statement and Technical Challenges

GitHub Repository

Project Link:
https://github.com/Aryan07175/vision-count-ai

Problem Statement

Traditional people-counting systems used in malls, classrooms, offices, and public spaces often suffer from a major issue known as **double counting**. Existing systems usually rely only on motion tracking or object detection models. Because of this, when a person leaves the camera frame and re-enters again, the system treats them as a completely new individual and increases the count again.

This leads to:

- Incorrect people count
- Inaccurate analytics
- Poor attendance estimation
- Inefficient crowd management
- Reduced reliability in surveillance systems

To solve this issue, we developed **VisionCount AI**, an AI-powered real-time human detection and unique people counting system. The system uses computer vision and facial recognition to assign a unique identity to every detected person. If the same individual reappears in the frame, the system recognizes them and avoids duplicate counting.

The solution is implemented as a web-based application using:

- React and Vite for frontend
- TensorFlow.js and COCO-SSD for real-time human detection
- FastAPI backend
- DeepFace and OpenCV for facial recognition and identity matching

The system performs real-time tracking directly through the webcam and maintains unique identities for detected users.

Existing Problem in Traditional Systems

Most existing human detection systems face the following limitations:

1. Double counting of the same person
2. Loss of tracking when a person exits the frame
3. Inability to remember previously detected individuals
4. Dependence only on object tracking instead of identity recognition
5. Poor performance in real-time low-cost camera systems
6. Inaccurate counting in crowded environments
7. Difficulty handling re-entry scenarios

Traditional tracking models such as:

- YOLO tracking
- DeepSORT
- ByteTrack
- OpenCV centroid tracking

mainly focus on movement tracking rather than persistent identity recognition.

Proposed Solution

VisionCount AI solves these issues by combining:

- Human detection
- Facial recognition
- Unique identity assignment
- Re-identification logic

Workflow:

1. Human detected through webcam
2. Snapshot sent to backend
3. DeepFace extracts facial embeddings
4. System compares embeddings with stored identities
5. Existing user → no increment in count
6. New user → assign new ID and increase counter

This approach significantly reduces duplicate counting and improves counting accuracy.

The proposed solution has been successfully implemented as a web application using React, TensorFlow.js, FastAPI, OpenCV, and DeepFace. The system supports:

- Real-time webcam detection

- Human tracking
- Persistent identity recognition
- Unique counting mechanism
- AI-based facial verification
- Real-time dashboard interface

The project repository is available at:
https://github.com/Aryan07175/vision-count-ai

Technical Challenges Faced

1. Real-Time Human Detection

Real-time detection requires continuous frame processing from the camera. Processing multiple frames per second creates high computational load and affects performance on low-end systems.

Challenges:

- FPS drops
- Delayed tracking
- Browser rendering lag
- CPU overload

---

2. Face Recognition Accuracy

The facial recognition system depends heavily on:

- Lighting conditions
- Camera quality
- Face angle
- Occlusion
- Blur

Problems occur when:

- Person turns sideways
- Face is partially hidden
- Low-light environment exists
- Multiple faces appear together

---

3. Camera Hardware Limitations

The system heavily depends on camera quality.

Low-end cameras create issues such as:

- Motion blur
- Low resolution
- Poor facial detail
- Frame skipping
- Incorrect identity matching

Different devices provide different camera qualities, making system behavior inconsistent.

Problems Faced While Converting React Web Application into Mobile Application

During the conversion of the React-based web application into an Android mobile application, multiple technical and hardware-related issues were encountered.

1. Mobile Camera Compatibility Issues

Unlike desktop browsers, Android devices use different camera APIs and hardware implementations.

Issues observed:

- Camera feed inconsistency
- Camera permission handling problems
- Delayed camera initialization
- Frame synchronization issues
- Different behavior across different Android devices

Some low-end devices were unable to maintain stable real-time camera streams.

---

2. Real-Time Processing Performance Issues

The application performs continuous:

- Human detection
- Frame processing

- Facial recognition
- Identity matching

On mobile devices, this created:

- Laggy camera feed
- Delayed tracking
- Reduced FPS
- Slow response time
- Overheating issues

Real-time AI inference requires higher computational capability than many mobile devices can provide.

---

3. GPU and Hardware Limitations

The solution requires stable GPU support for:

- AI model execution
- TensorFlow processing
- Real-time rendering
- Continuous frame analysis

However, many Android devices:

- Lack dedicated GPU acceleration
- Have limited memory resources
- Cannot efficiently handle AI workloads

As a result:

- Performance instability occurs
- App freezing issues appear
- AI processing becomes slow
- Detection accuracy decreases

---

4. Device Camera Quality Variations

Different Android devices provide different camera quality levels.

Problems faced:

- Low-resolution frames
- Motion blur

- Poor low-light performance
- Unstable autofocus
- Weak image clarity

Since the proposed solution depends heavily on facial recognition, low-quality cameras negatively affect identity matching accuracy.

---

5. Application Crashes During Processing

Continuous real-time frame processing increased memory and CPU usage.

This caused:

- Random application crashes
- Camera stream interruption
- App freezing during long usage
- Increased battery consumption
- Device heating

Some low-end Android devices were unable to handle simultaneous AI processing and camera streaming.

Problems Faced While Converting the System into an Android Application Using Java

During the process of converting the web-based system into a mobile application using Java, several technical limitations were identified.

1. OpenCV and AI Integration Issues in Java

The project heavily relies on:

- DeepFace
- OpenCV
- TensorFlow.js
- Python-based AI models

These technologies are not naturally optimized for Java-based Android development.

Issues faced:

- Complex OpenCV integration
- Difficult AI model compatibility
- Lack of stable DeepFace support in Java
- Increased implementation complexity
- Poor real-time performance

Most facial recognition libraries are primarily designed for Python environments.

---

2. Android Device Hardware Variability

Android devices differ significantly in:

- Camera quality
- CPU performance
- GPU capability
- RAM availability

Because of this:

- The application behaves differently across devices
- Detection speed becomes inconsistent
- Recognition accuracy drops on low-end phones

Low-level cameras cannot provide stable image quality required for accurate facial recognition.

---

3. GPU Processing Limitations

The proposed solution requires significant computational power for:

- Real-time frame processing
- AI inference
- Facial embedding generation
- Identity matching

Many Android devices lack:

- Dedicated GPU acceleration
- Stable AI processing support
- Sufficient memory bandwidth

As a result:

- App crashes may occur
- Frame lag increases
- Camera feed freezes
- Real-time processing becomes unstable

---

4. Mobile Camera Stream Instability

Unlike browsers running on laptops with stable webcam drivers, Android camera systems vary heavily by manufacturer.

Problems observed:

- Camera initialization failure
- Inconsistent frame rate
- Delayed frame capture
- Camera permission conflicts
- App crashes during camera switching

---

5. Resource Consumption

Real-time AI processing consumes:

- High CPU usage
- Large memory allocation
- Continuous frame buffering

This causes:

- Battery drain
- Device heating
- Performance throttling
- Reduced application stability

Conclusion

VisionCount AI successfully addresses the problem of duplicate counting in traditional human detection systems by introducing AI-powered identity recognition and persistent tracking.

The system demonstrates how facial recognition combined with real-time human detection can significantly improve counting accuracy and reduce duplicate entries.

However, deploying such a system on Android devices using Java introduces several hardware and software limitations due to:

- Limited GPU capability
- Device camera variability
- AI framework compatibility issues
- Real-time processing constraints

Therefore, the web-based architecture currently provides a more stable and efficient environment for the proposed solution.